Paper Review: *An End-to-End Transformer Model for 3D Object Detection*

Nemanja Vujadinović
ENS Paris-Saclay, Gif-sur-Yvette

1 Introduction

Object detection is a fundamental task in computer vision whose goal is to identify and localize objects in visual data by drawing class-labeled bounding boxes around them [20]. This task is important to a wide range of applications, from autonomous vehicles and robotics to surveillance and augmented reality [4].

Traditional object detection methods typically operate on 2D image data [12, 8, 13]. While these methods have achieved strong performance, they provide only limited information about the spatial structure of a scene (e.g., objects' size and volume), since the predicted bounding boxes are defined in image coordinates.

To overcome these limitations and better localize objects within a spatial environment, researchers have increasingly focused on 3D object detection. Unlike in 2D detection, 3D methods often operate on point clouds. Compared to images, point clouds are irregular and unordered, which makes them more challenging to process. Many existing approaches [9] attempt to address this by incorporating engineered inductive biases and specialized 3D operations, which often result in complex pipelines.

In their paper, Misra et al. propose 3DETR (3D DEtection TRansformer), an end-to-end transformer-based model for 3D object detection that eliminates the need for these hand-crafted components. The method aims to simplify existing detection pipelines while maintaining competitive performance.

This report provides a structured review of the paper. The first section summarizes the paper's positioning relative to prior work, its main contributions, methodology, and reported results. The second section presents our own experiments, which cover both reproductions of the paper's results and additional analyses. The third section offers a critical assessment of the paper's strengths and weaknesses, grounded in both the literature and our experimental findings.

2 Paper Summary

2.1 Positioning to the State of the Art

As already noted, 3D object detection aims to identify and localize objects from 3D data. Such data can be represented in several forms, including voxels or meshes. However, constructing these representations often requires additional post-processing and may introduce information loss due to quantization. For this reason, many modern approaches operate directly on point clouds. The foundation for processing raw point clouds was established by the PointNet [10] and PointNet++ [11] architectures. These models demonstrated that point clouds can be processed directly using neural networks while preserving permutation invariance. Still, they were originally designed for classification and segmentation tasks rather than object detection.

Building on PointNet++, VoteNet [9] introduced a strong baseline for 3D object detection. This method uses a Hough voting mechanism to aggregate point features and predict object centers. Several subsequent works extended this framework to further improve detection performance. For example, H3DNet [19] incorporates additional geometric reasoning, while HGNet [2] improves the voting process using a hierarchical graph network with feature pyramids. Despite their strong results, these approaches often rely on hand-crafted voting mechanisms, geometric priors, and multi-stage pipelines, which makes the resulting models complex and both difficult to tune and generalize across datasets.

Transformers provide a promising alternative to these designs due to their self-attention mechanism. Self-attention is permutation-invariant and capable of capturing long-range relationships between elements in the input, which makes it well suited for processing unordered point clouds. In the context of 2D object detection, DETR [1] demonstrated that a standard encoder-decoder transformer can replace several hand-crafted components, such as anchor boxes and non-maximum suppression (NMS), while maintaining competitive performance. Inspired by this idea, 3DETR extends the transformer-based detection paradigm to the 3D domain by applying a transformer architecture directly to point cloud representations and removing the need for task-specific operators and hand-crafted priors.

2.2 Main contributions

The main contribution of the paper is the proposal of a novel and simple approach for 3D object detection using a transformer-based model. Through end-to-end training, set prediction with Hungarian matching [7], and non-parametric queries, the method removes the need for handcrafted components such as voting, separate training stages, and NMS. The paper also introduces non-parametric queries, which are better suited for irregular point clouds and allow flexible query counts at inference, together with Fourier positional embeddings [16] that improve spatial encoding over standard positional encodings. In addition, the method is designed as a flexible and modular architecture, where individual components can be replaced and adapted to different tasks, such as classification, with the goal of building a framework that can serve as a future extensible backbone for 3D vision tasks. Finally, the work provides a comprehensive empirical evaluation, including comparisons to prior methods, detailed ablation studies on architectural components, and analysis of design choices such as the number of queries, encoder and decoder depth, and the role of NMS.

2.3 Methodology

In broader terms, the 3DETR model takes a point cloud as input, processes it using an end-to-end transformer, and predicts a set of 3D bounding boxes corresponding to objects in the scene. The following paragraphs describe each stage of this pipeline.

Set Abstraction. The first step of the pipeline processes the raw input point cloud. Since point clouds can contain a very large number of points, feeding them directly into a transformer is computationally expensive because the attention mechanism scales quadratically with the number of points. For this reason, the point cloud is first reduced using the set abstraction strategy inspired by PointNet++.

The reduction begins with the farthest point sampling (FPS) [11]. Here, points are selected iteratively so that each newly selected point is the farthest from all previously selected points. This encourages the sampled points to be spread across the entire scene instead of being concentrated in dense regions. After this step, the point cloud is reduced to $N' = 2048$ points. The sampling is then followed by a set aggregation. In this stage, each of the N' points is passed through an MLP that produces a feature vector of dimension $d = 256$.

Transformer Encoder. The resulting $N' \times d$ feature vectors of the set abstraction stage represent the encoded point cloud and serve as the input to the transformer encoder. In its base configuration, the encoder consists of three standard transformer encoder layers. The attention mechanism in these layers is standard and does not include any task-specific modifications. Positional embeddings are not used because point coordinates are already incorporated into the features during the previous stage.

In addition to the base 3DETR model, the authors also propose a variant called *3DETR-m*. Compared to the base model, 3DETR-m uses the same overall architecture but modifies the encoder to introduce an inductive bias toward local geometric structure. The motivation for this design comes from PointNet++, where local neighborhoods are considered more informative than distant points when extracting features. Transformers allow an easy adaptation of this idea by applying a binary mask to the self-attention mechanism. The mask restricts each point to attend only to other points within a predefined spatial radius, which mimics neighborhood aggregation. In the 3DETR-m setup, an additional downsampling step is first applied to reduce the number of points from N' to $N'' = 1024$. The attention mechanism in each of the three encoder layers is then restricted using a spatial mask with radii of [0.16, 0.64, 1.44]. As a result, the first layer focuses on a very local structure, while later layers gradually increase their receptive field. This design follows a similar intuition to CNNs, where early layers capture simple local patterns, and deeper layers provide more

global context.

In both its forms, the encoder produces $N' \times d$ contextualized point features that are passed to the decoder.

Query Embeddings. Alongside the features from the encoder, the decoder also takes query embeddings as input. Query embeddings were introduced in the DETR model as vectors that learn to detect objects during training. These queries were randomly initialized and optimized during training. Building on this idea, 3DETR proposes non-parametric query embeddings, meaning that they are predefined and derived directly from the input data before being passed through the architecture.

To construct query embeddings, B points are first sampled from N' points using FPS, where B is a hyperparameter. Each point is then converted into a query embedding by applying Fourier positional encoding to its 3D coordinates, followed by an MLP projection. The resulting embeddings therefore represent candidate object locations in the scene. Intuitively, since each query pre-encodes a spatial location, the decoder can focus on predicting whether an object exists at that location and which object it is, rather than learning object proposals entirely from scratch, as in the case of DETR.

Transformer Decoder. As explained, the decoder takes two inputs: the $N' \times d$ features produced by the encoder and the $B \times d$ query embeddings. In its base configuration, the decoder consists of eight standard transformer decoder layers, each containing self-attention, cross-attention, and a feed-forward network. Self-attention operates over the query embeddings, which allows them to communicate with each other and helps the model avoid predicting duplicate bounding boxes. Cross-attention, on the other hand, allows each query to attend to the encoded point features and gather contextual information about the scene. Finally, the decoder outputs a set of B features.

Bounding Box Prediction. In the final stage of the architecture, the query features produced by the decoder are passed through separate MLP heads to predict four parameters of a bounding box:

- Location c: The model predicts offset Δq and computes the center of the box as $c = q + \Delta q$, where q is the corresponding query coordinate.
- Size d: The model predicts dimension $d \in [0, 1]^3$, which defines the width, height, and depth of the box.
- Orientation a: The model classifies the rotation angle a_c into 12 discrete bins and then estimates a residual value a_r within the predicted bin to determine the final continuous angle.
- Semantic class s: The model predicts a probability vector s across K object classes with an additional background class for non-object queries.

Given these parameters, each bounding box can be represented as $b = [c, d, a, s]$.

Set-to-Set Matching and Loss. Once the final bounding box predictions are obtained, the training loss can be computed. However, since the decoder outputs query features with no predefined ordering (*i.e.,* set prediction), the predicted boxes must first be matched with the ground truth boxes. To achieve this, the model uses bipartite matching with the Hungarian algorithm, which finds the optimal one-to-one assignment between predicted boxes $\hat{\mathbf{b}}$ and ground truth boxes $\mathbf{b}$ by minimizing a matching cost:

$$C_{\text{match}}(\hat{\mathbf{b}}, \mathbf{b}) = \underbrace{-\lambda_1 \,\text{GIoU}(\hat{\mathbf{b}}, \mathbf{b}) + \lambda_2 \|\hat{\mathbf{c}} - \mathbf{c}\|_1}_{\text{geometric}} + \underbrace{-\lambda_3 \hat{s}[s_{gt}] + \lambda_4 (1 - \hat{s}[s_{bg}])}_{\text{semantic}}.$$

Here, the geometric term accounts for the box overlap and the distance between box centers, while the semantic term measures the likelihood of assigning the correct class and whether the prediction corresponds to the background. It is important to note that, since this is one-to-one matching, some boxes may not be matched to any ground truth one, in which case they are assigned to the background class.

Once the matches are obtained, the model is trained using a loss function that combines losses for all predicted box parameters:

$$\mathcal{L}_{\text{3DETR}} = \lambda_c \|\hat{\mathbf{c}} - \mathbf{c}\|_1 + \lambda_d \|\hat{\mathbf{d}} - \mathbf{d}\|_1 + \lambda_{\text{ang}} \|\hat{\mathbf{a}}_r - \mathbf{a}_r\|_{\text{huber}} - \lambda_{ac} \mathbf{a}_c^\top \log \hat{\mathbf{a}}_c - \lambda_s \mathbf{s}_c^\top \log \hat{\mathbf{s}}_c.$$

Here, the box center and dimensions are optimized using the ℓ_1 loss, the angle residual is optimized using the Huber loss, while the angle class and semantic class are optimized using cross-entropy loss. For matched

predictions, all loss terms are computed. For unmatched predictions, only the semantic classification loss is applied, which pushes them toward the background class.

For axis-aligned datasets (e.g., ScanNetV2), the authors additionally include a GIoU [14] loss term. This helps penalize cases where individual parameter errors are small, but the predicted box still has poor overlap with the ground truth. For non-axis-aligned datasets, this term is not used.

Finally, during training, predictions from every decoder layer are used to compute the loss, which is then averaged across layers. In contrast, during inference, only the predictions from the final layer are used.

2.4 Validation and Results

To evaluate the 3DETR and 3DETR-m models, for 3D object detection, the authors measure mean Average Precision (mAP) at IoU thresholds of 0.25 (AP_{25}) and 0.50 (AP_{50}) on two indoor benchmarks: ScanNetV2 and SUN RGB-D. For ScanNetV2, scenes are downsampled to 40K points with axis-aligned bounding boxes across 18 categories. SUN RGB-D uses 20K points per sample with oriented bounding boxes and is evaluated on the 10 most frequent categories. Moreover, the paper employs a different number of object queries for each dataset, using 256 queries for ScanNetV2 and 128 for SUN RGB-D.

The models are compared against VoteNet and its non-voting counterpart, BoxNet [9], as conceptually similar approaches, as well as H3DNet, which serves as a state-of-the-art reference. Notably, both VoteNet and BoxNet baselines were further tuned through hyperparameter optimization to ensure a fair comparison.

	ScanNetV2		SUN RGB-D	
Method	AP_{25}	AP_{50}	AP_{25}	AP_{50}
BoxNet	49.0	21.1	52.4	25.1
VoteNet	60.4	37.5	58.3	33.4
H3DNet	67.2	48.1	60.1	39.0
3DETR	62.7	37.5	58.0	30.3
3DETR-m	65.0	47.0	59.1	32.7

Table 1: Detection performance on ScanNetV2 and SUN RGB-D.

As shown in Table 1, 3DETR significantly outperforms BoxNet on both datasets and achieves performance comparable to VoteNet. The impact of introducing inductive bias is evident in the results of 3DETR-m, which consistently improves over the base model and surpasses BoxNet in all, and VoteNet in most settings, except on SUN RGB-D at AP_{50}. While both 3DETR variants remain below the performance of H3DNet, the gap is relatively modest given the simplicity of the architecture.

Overall, these results demonstrate that a standard transformer-based detector, with minimal 3D-specific design, can effectively compete with more heavily engineered 3D detection methods, which is the central claim of the paper.

Additional Analysis and Design Choices. The authors further analyze the contribution of individual architectural components through a series of ablation experiments. First, replacing the transformer encoder with a standard PointNet++ backbone results in a modest drop in performance (a decrease of 3.1% in AP_{25} on ScanNetV2), which suggests that the 3DETR's encoder contributes meaningfully beyond simple feature extraction. In contrast, removing the decoder and relying on a fixed proposal mechanism leads to a more significant degradation, which the authors interpret as evidence that cross-attention is responsible for grounding queries to relevant scene regions. The effect of NMS is also examined, with only a minor performance drop observed when it is removed, which highlights the robustness of the proposed architecture and training objective to it. Additionally, the authors show that both Fourier positional encoding and non-parametric query embeddings are critical for high detection performance. Increasing the number of encoder and decoder layers further improves results up to a certain point, after which gains saturate. They also demonstrate that the model can be adapted to different computational budgets by reducing the number of decoder layers and object queries. Finally, the encoder is further shown to generalize beyond the detection task. When applied to 3D shape classification, it achieves competitive performance compared to architectures specifically designed for 3D tasks.

3 Our Experiments

In this section, we build upon the existing 3DETR repository[1] to either reproduce the results and experiments reported in the paper or explore additional ones. For experiments involving VoteNet, we additionally rely on its official implementation[2]. The outcomes of these experiments will later be used to highlight the strengths and limitations of the proposed 3DETR method. The details on the implementation are shown in the Appendix 5.

3.1 Experimental Setup

The following experiments on 3D object detection are conducted on the ScanNetV2 validation split. We do not use the SUNRGB-D dataset, as the provided scripts and explanations[3] did not work as expected. Specifically, approximately 25-30% of the validation data resulted in errors when processed with the provided MATLAB scripts, which substantially reduced the usable portion of the dataset. Therefore, to ensure fair and consistent experiments, we decided to exclude this dataset. We use the official 1080-epoch pre-trained models provided within the repository. All experiments were run on an RTX 3090 GPU.

3.2 Reproduction of Reported Results

Here, we aim to reproduce a subset of the experiments reported in the paper. While quantitative 3D detection results can be directly evaluated using the available implementation, several analyses described in the paper are not explicitly provided in the official codebase. We therefore re-implement these experiments to verify the corresponding claims and further analyze the method.

3D Detection Performance. As noted, the authors report that 3DETR achieves strong detection performance on ScanNetV2. We reproduce these results and report them in Table 2. The obtained values closely match those reported in the paper, with differences of at most 1.16% in AP_{25}. These minor differences stem from stochastic components such as FPS, which lead to variations across runs, as noted in the official repository.

Method	AP_{25}	AP_{50}
3DETR (ours)	61.69	37.77
3DETR (original)	62.7	37.5
3DETR-m (ours)	64.42	45.84
3DETR-m (original)	65.0	47.0

Table 2: Reproduction of 3DETR and 3DETR-m detection performance on ScanNetV2.

In the paper, the authors present qualitative results only on the SUN RGB-D dataset. Here, we instead evaluate qualitative performance on ScanNetV2. As shown in Fig. 1, 3DETR successfully detects multiple objects, with bounding boxes that are well localized and capture object extent accurately. This observation is in line with the previously reported quantitative results.

However, in addition to strong results, we also observe several recurring failure cases, mostly in terms of false positives. First, Fig. 2 (a) shows a pattern where the model detects objects that are not part of the label set and therefore not present in the ground truth. Specifically, the model here detects a basket and classifies it as a garbage bin. While this can be seen as a meaningful qualitative behavior, it is quantitatively incorrect and reduces performance. Next, Fig. 2 (b) shows another failure where the model predicts spurious bounding boxes in empty or unobserved regions of the scene, often near gaps between surfaces or areas with missing geometry. In Fig. 2 (c), we observe a common failure where the model fails to detect doors, which is a pattern that also extends to windows and wall-mounted objects. Finally, Fig. 2 (d) presents a repetitive error where the model mistakenly detects parts of walls as doors. More generally, we observe that regions near the boundaries of the scene, where surfaces are incomplete or partially scanned, often lead to incorrect detections. Overall, the results suggest that the model struggles with thin structures and ambiguous surfaces, which are less explicitly defined compared to more distinctive objects such as a chair or bed.

[1] https://github.com/facebookresearch/3detr
[2] https://github.com/facebookresearch/votenet
[3] https://github.com/facebookresearch/votenet/tree/main/sunrgbd

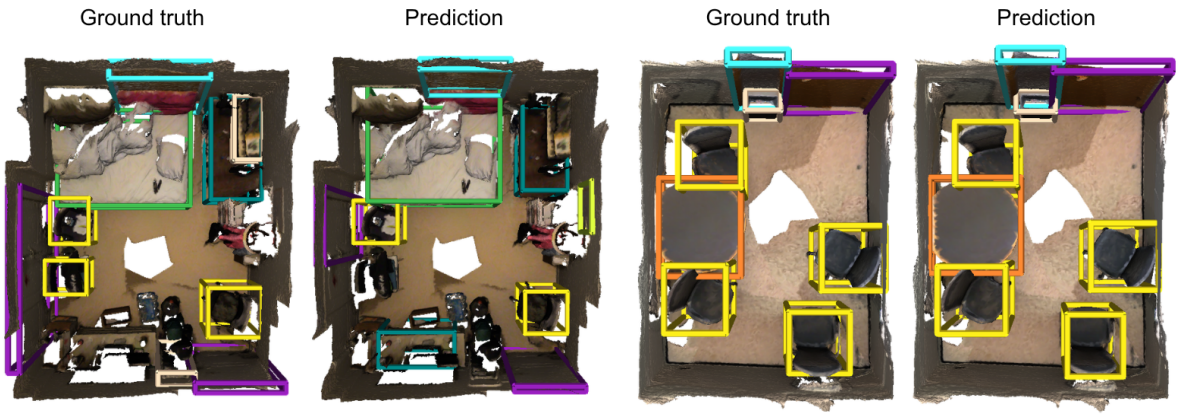


Figure 1: Successfull qualitative results of 3DETR on ScanNetV2.

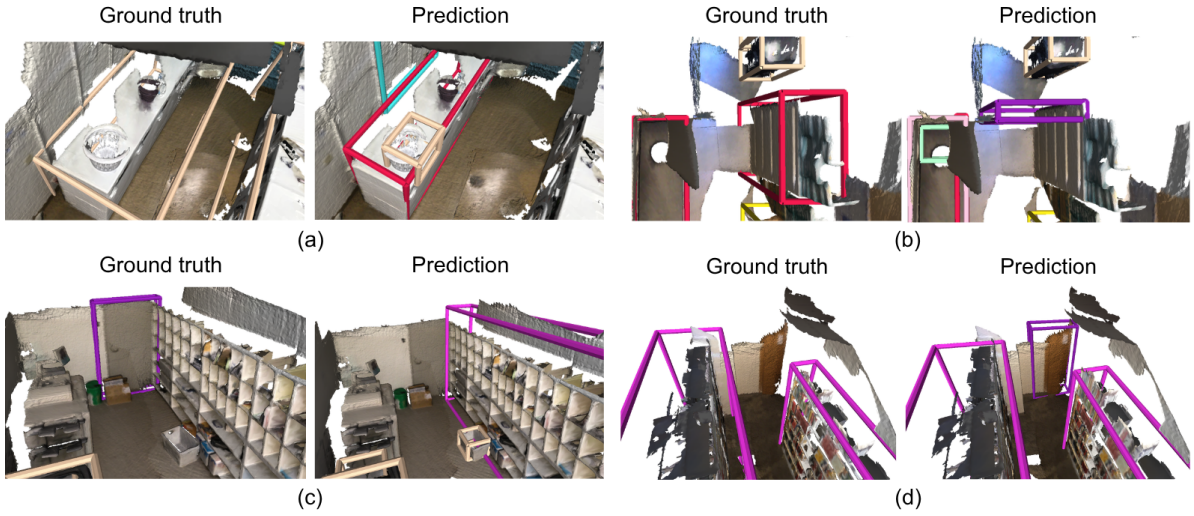


Figure 2: Recurring failure cases of 3DETR on ScanNetV2.

Effect of the Number of Queries on Performance and Inference Time. The authors claim that an advantage of the proposed model is the ability to adapt computation at inference time by varying the number of queries. We reproduce this experiment to evaluate the practical impact of this flexibility. Table 3 reports detection performance and mean inference time as a function of the number of queries for 3DETR and the number of proposals for VoteNet. The results primarily indicate that 3DETR is highly sensitive to this parameter: with only 32 queries, AP_{25} drops to 13.16, which is significantly lower than VoteNet's 44.03 at the same setting. On the other hand, to reach performance comparable to or better than VoteNet, 3DETR requires at least 256 queries. This suggests that 3DETR relies on a sufficiently large set of queries to adequately cover the scene, whereas VoteNet benefits from stronger spatial priors that allow it to perform well even with fewer proposals. In terms of inference time, 3DETR is consistently slower than VoteNet across all settings, and the gap increases as the number of queries grows. In conclusion, while 3DETR can match or exceed VoteNet's accuracy, it does so at a higher computational cost and with a stronger dependence on the query budget. Therefore, although the model allows adaptation to different computational budgets, this flexibility comes with a significant trade-off between efficiency and performance.

Method	#Queries/Proposals	AP		Time (ms)
		AP_{25}	AP_{50}	Mean Time
VoteNet	32	44.03	25.03	105.36
	64	52.75	31.46	106.58
	128	57.32	33.70	109.12
	256	58.27	33.87	120.94
	512	57.67	34.27	122.86
3DETR	32	13.16	4.90	128.41
	64	34.25	16.59	129.45
	128	53.06	30.20	132.66
	256	61.69	37.77	147.38
	512	63.98	39.43	161.90

Table 3: Impact of number of queries (3DETR) and proposals (VoteNet) on detection performance and time.

An additional consideration is that ScanNetV2 is an indoor dataset, where scenes are relatively structured and dense, and object localization is easier. In more challenging settings, such as outdoor scenes with larger spatial extent and sparser point distributions, a higher number of queries would likely be required, which would further increase computational cost. We investigate this aspect in more detail in the fourth section.

NMS Effect on Performance. The authors claim that 3DETR is robust to the removal of NMS due to its training objective, unlike methods such as VoteNet. We here reproduce this experiment and observe only a minor decrease in 3DETR's performance when NMS is removed, as shown in Table 4. In contrast, VoteNet exhibits a significant drop in accuracy, which highlights its strong dependence on NMS. This confirms that the 3DETR model inherently reduces duplicate predictions through one-to-one set matching and is largely robust to the absence of post-processing. In addition, the results also prove that 3DETR's loss formulation effectively mitigates the need for specialized 3D loss functions, which are commonly used in prior work.

Method	NMS	AP_{25}	AP_{50}
VoteNet	None	9.76	6.17
	3D NMS	58.27	33.87
3DETR	None	60.82	37.96
	2D NMS	58.61	36.88
	3D NMS	61.69	37.77

Table 4: Effect of different NMS configurations on VoteNet and 3DETR detection performance.

Encoder for Classification. The authors suggest that the 3DETR encoder can generalize beyond detection to tasks such as classification. To examine this claim, we adapt the encoder for 3D shape classification on ModelNet40. In order to, perform classification, we first need to introduce a global representation of the input points because the base encoder does not generate one. In 3DETR, this is done by creating and prepending a CLS token to the point features before passing them through the encoder, and using its output as input to a two-layer MLP classifier. In 3DETR-m, instead of using CLS, we apply max pooling over the final encoder features and feed the resulting representation into the same classifier. All other components stay the same. We evaluate the classification performance using mean class accuracy (mAcc) and overall accuracy (OA). Due to computational constraints, we only train the 3DETR variant for 250 epochs.

Table 5 presents our classification results together with several reference methods. Our results, with the best performance achieved at epoch 212, are slightly lower than those reported in the paper, which we attribute to the aforementioned randomness as well as training details not fully specified by the authors. Despite this gap, the results confirm that the 3DETR encoder generalizes well to classification: it outperforms PointNet in terms of mAcc and remains competitive with more advanced methods such as DGCNN [17]. This indicates that the design choices behind 3DETR can be applied to and are useful for other tasks.

Method	OA	mAcc
PointNet	89.2	-
PointNet++	91.9	-
DGCNN	92.2	90.2
3DETR (paper)	92.1	89.1
3DETR (ours)	89.8	88.3

Table 5: 3D shape classification performance of various methods on ModelNet40.

However, training the 3DETR encoder for classification is substantially slower than lightweight alternatives such as PointNet, with each epoch requiring significantly more time (212s compared to 9s). While the results confirm the encoder's generalization ability, the approach has limited practical applicability and notable cost.

3.3 Beyond the Paper Results: Additional Analysis

Recall Analysis Across Object Size and Point Count. In this experiment, we stratify objects into equal-frequency bins based on their volume and point count to analyze recall under specific conditions. As shown in Fig. 3, recall is significantly lower for the two smallest volume bins, while it quickly stabilizes across larger ranges. This indicates that the model struggles particularly with smaller objects. A similar trend is observed when grouping by point count, although the performance gap between bins is notably smaller.

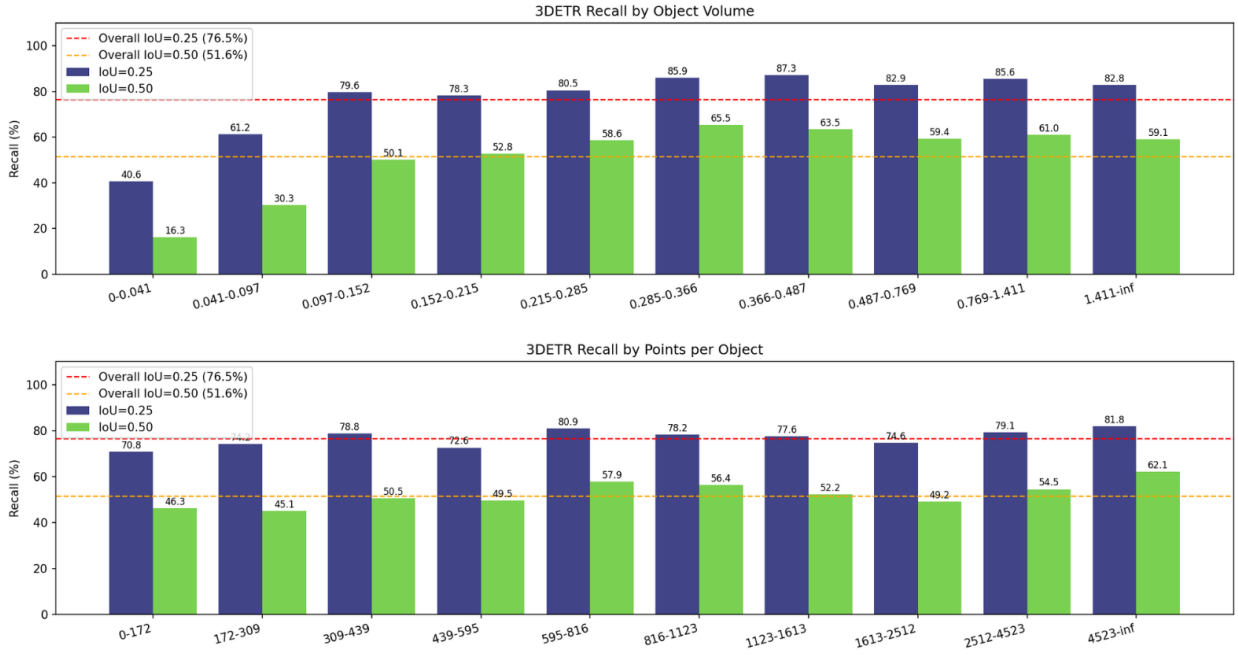


Figure 3: 3DETR recall binned by equal percentiles of object volume (top) and point count (bottom).

There are several factors that may explain this behavior. First, small objects are impacted by the sensitivity of IoU-based evaluation, where even minor localization errors lead to a significant drop in overlap, and therefore cause otherwise reasonable predictions to be counted as false negatives. Next, the initial point cloud downsampling significantly reduces the number of points, which disproportionately affects small or sparse objects. As a result, they may lose important geometric details early in the pipeline and become further underrepresented. In addition, since queries are sampled using FPS, coverage of such regions is not guaranteed, and in practice, they may be only weakly represented or not represented at all. Finally, during cross-attention, queries associated with these regions may be dominated by nearby larger structures, which weakens their features and makes them harder to detect.

Cross-attention Visualization. In the paper, the authors visualize encoder attention and interactions between queries, but do not analyze cross-attention between queries and encoder points. To better understand this relationship, we extract attention weights from the last three decoder layers, average them, and visualize the resulting cross-attention maps in Fig. 4. In the top row, we observe correctly localized objects whose corresponding points receive relatively high-attention values. However, these high values are also distributed across other regions of the scene. This effect is even more evident in Fig. 4 (c), where the detected bed corresponds to only a small subset of high-attention points, while many of the highest ones appear in unrelated regions such as walls and doors. The pattern reaches an extreme in Fig. 4 (d), where the detected toilet receives practically no concentrated attention, with high values almost entirely distributed in unrelated regions of the scene.

These observations reveal that, in practice, 3DETR's queries often attend to points that are spatially distant from the target objects. As a consequence, the model's behavior deviates from the principle of locality in object detection, where predictions are expected to rely primarily on features within the object's spatial support rather than the entire scene.

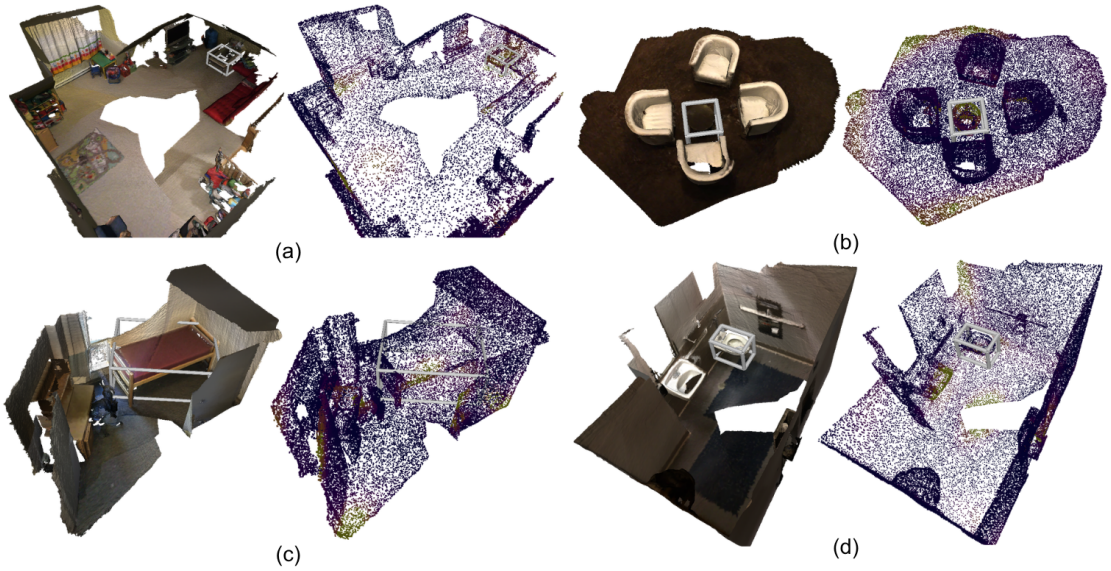


Figure 4: Per-query cross-attention maps averaged over the last three decoder layers.

4 Critical Assesment

4.1 Strengths

The main strength of the paper lies in proposing a novel transformer-based design for 3D object detection that removes the need for handcrafted components. As reported by the authors and further validated by our experiments, although the method is partly positioned as a proof-of-concept, it nevertheless achieves competitive or superior performance compared to prior approaches such as VoteNet and its non-voting variant BoxNet. Its simple design built on standard transformer blocks, when combined with appropriate choices such as Fourier positional encoding, non-parametric queries, and end-to-end set-based training, demonstrates that strong performance can be achieved without heavy geometric engineering, which is a direction that has since been further developed in subsequent work. In addition, the use of bipartite matching and a set-based loss formulation reduces dependence on post-processing heuristics, which we confirm through experiments showing that performance degrades only marginally when removing NMS, in contrast to VoteNet. Furthermore, the encoder-decoder design proves to be flexible and transferable, as demonstrated both in the paper and in our experiments, where the encoder component generalizes beyond detection to classification tasks. This flexibility is further supported by 3DETR-m variant, which incorporates additional 3D inductive bias

while maintaining the same overall framework and achieving improved performance. Together, these observations highlight the potential of 3DETR as a strong and extensible baseline for future work, in a similar way that VoteNet has been extended by subsequent H3DNet and HGNet methods.

4.2 Weaknesses & Limitations

The first limitation of the 3DETR method is its computational cost and training time. The authors report that training on SUN RGB-D requires approximately 34–36 hours[4], compared to around 10 hours for VoteNet [9]. Our experiments, although designed to evaluate favorable aspects of the model, reflect a similar trend. Specifically, when varying the number of queries, we observe that VoteNet consistently achieves lower inference time across all query/proposal configurations. Moreover, for 3DETR to reach comparable or better performance, it requires a higher number of queries, which further increases computational cost. This overhead is also evident in our classification experiment, where the encoder remains significantly slower than PointNet, despite providing only marginal performance improvements. These limitations arise from the transformer architecture itself and its attention mechanism, and would become even more pronounced with a larger number of input points to the encoder and, as discussed, a higher number of queries.

The second limitation is the lack of evaluation on outdoor benchmarks, making it unclear how the model generalizes beyond indoor scenes. Since we were unable to train the model in this setting due to computational constraints and time limits, we rely on our recall analysis to infer potential behavior. As shown earlier, the model struggles significantly with smaller objects and shows reduced performance for objects with fewer points, which we attributed to aggressive downsampling and query initialization via FPS. Notably, such downsampling is unavoidable, as 3DETR operates directly on point clouds and self-attention scales quadratically with the number of points. Despite this, overall performance remains high on indoor datasets, because these scenes are typically dense and well-covered, with relatively few small or sparse objects. In contrast, outdoor scenes are large-scale, sparse, and unevenly distributed, where objects are often small relative to the scene and represented by a limited number of points. Under these conditions, the failure modes observed in our analysis are expected to be further amplified, as objects may be even more undersampled or insufficiently covered by queries. As a result, the reduced set of sampled points and query initialization strategy may fail to adequately cover object regions, which would lead to degraded performance. This observation is consistent with subsequent work [5], which highlights that for large-scale outdoor point clouds, the scale and coverage of sampled points in 3DETR become a critical limitation.

The final limitation we identify comes from our cross-attention visualization experiment, where we observe that queries distribute their attention across the entire scene, and therefore weaken the locality. This suggests that the model lacks an appropriate inductive bias, which ultimately impacts its overall performance.

4.3 Recommendations and Subsequent Work

Although the paper provides a series of ablations on design choices and evaluation, which represent one of its main contributions, the work would be strengthened if the official repository included scripts for reproducing these experiments (e.g., classification and visualization). This would enable an easier verification of the claims, but would also facilitate further analysis. Moreover, as discussed, both our findings and observations from subsequent work suggest that the model is likely to perform poorly on outdoor benchmarks. However, to the best of our knowledge, the original 3DETR model has not been explicitly evaluated in this setting. Conducting such experiments would therefore provide valuable insights into its generalization capabilities.

Regarding the aforementioned limitations, subsequent work building on the 3DETR approach has addressed and mitigated several of them. Concerning the computational cost, $\mathcal{H}$-DETR [6] proposes one-to-many matching during training to improve the training efficacy of positive samples and ultimately achieve faster convergence. Next, methods such as Uni3DETR [18], Voxel Set Transformer [5], and Li3DeTr [3] replace point-level representations with voxel features by aggregating points into a 3D grid and performing attention over them. This reduces the number of tokens and improves computational efficiency, which ultimately enables detection in large and sparse outdoor scenes. Lastly, V-DETR [15] tackles the locality issue by computing positional encodings based on each point's relative position to 3D bounding boxes, which allows

[4] https://github.com/facebookresearch/3detr/issues/4

the model to focus on points near the objects and improve spatial modeling. This modification led to significant improvements over the base model, with gains of 12.8% and 19% in AP25 and AP50, respectively.

5 Conclusion

In this work, we summarized and critically analyzed 3DETR, an end-to-end transformer-based model for 3D object detection on point clouds that requires fewer 3D-specific design choices and hyperparameters than prior methods. Its performance is competitive with approaches such as BoxNet and VoteNet, while remaining close to the state-of-the-art H3DNet. Despite its strengths, the method comes with notable limitations, including higher computational cost, lack of locality in detection, and unclear generalization to outdoor scenarios, where both our analysis and subsequent work suggest inherent limitations of the proposed design. Nevertheless, the paper introduces a novel and simplified architecture for 3D object detection, and more importantly, it establishes a flexible framework that can serve as a foundation for subsequent works, which build upon it to address these limitations and further improve detection performance.

References

[1] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers, 2020.

[2] Jintai Chen, Biwen Lei, Qingyu Song, Haochao Ying, Danny Z. Chen, and Jian Wu. A hierarchical graph network for 3d object detection on point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 392–401, 2020.

[3] Gopi Krishna Erabati and Helder Araujo. Li3detr: A lidar-based 3d detection transformer. *arXiv preprint arXiv:2210.15365*, 2022.

[4] Yalda Ghasemi, Heejin Jeong, Sung Choi, Kyeong-Beom Park, and Jae Yeol Lee. Deep learning-based object detection in augmented reality: A systematic review. *Computers in Industry*, 139:103661, 08 2022.

[5] Chenhang He, Ruihuang Li, Shuai Li, and Lei Zhang. Voxel set transformer: A set-to-set approach to 3d object detection from point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8417–8427, June 2022.

[6] Ding Jia, Yuhui Yuan, Haodi He, Xiaopei Wu, Haojun Yu, Weihong Lin, Lei Sun, Chao Zhang, and Han Hu. Detrs with hybrid matching, 2023.

[7] Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955.

[8] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. Ssd: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, 2016.

[9] Charles R. Qi, Or Litany, Kaiming He, and Leonidas J. Guibas. Deep hough voting for 3d object detection in point clouds, 2019.

[10] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation, 2017.

[11] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space, 2017.

[12] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[13] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[14] Hamid Rezatofighi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression, 2019.

[15] Yichao Shen, Zigang Geng, Yuhui Yuan, Yutong Lin, Ze Liu, Chunyu Wang, Han Hu, Nanning Zheng, and Baining Guo. V-detr: Detr with vertex relative position encoding for 3d object detection, 2023.

[16] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[17] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic graph cnn for learning on point clouds, 2019.

[18] Zhenyu Wang, Yali Li, Xi Chen, Hengshuang Zhao, and Shengjin Wang. Uni3detr: Unified 3d detection transformer, 2023.

[19] Zaiwei Zhang, Bo Sun, Haitao Yang, and Qixing Huang. H3dnet: 3d object detection using hybrid geometric primitives, 2020.

[20] Zhengxia Zou, Keyan Chen, Zhenwei Shi, Yuhong Guo, and Jieping Ye. Object detection in 20 years: A survey, 2023.

Appendix

A. Implementation Overview and Code Contribution

This appendix outlines the structure of the implementation and the additional components developed for the experimental analysis. For the 3D object detection experiments, we use the default parameters provided in the official codebase and described in the paper (e.g., 3 encoder layers, 8 decoder layers, a 2-layer MLP with 256 hidden dimensions, and others). For the classification task, we use a learning rate of 4×10^{-4} with a weight decay of 0.1, optimized using AdamW. The learning rate is linearly warmed up from 4×10^{-7} to the initial value over the first 20 epochs, and then decayed to 4×10^{-5} over the remaining 230 epochs.

Data Preparation.

- `data_helper/download_scannet.py` & `data_helper/download_scannet.ps1`: Scripts for downloading the ScanNet dataset, provided in both Python and PowerShell variants.
- `data_helper/rotate_validation_scans.py`: The script used for rotating the raw ScanNet `_vh_clean_2.ply` meshes to align with the coordinate frame expected by the detection data (*i.e.* consistent with the preprocessed `_bbox.npy` annotations). This is a one-time preprocessing step required before running visualization.

Inference Helper.

- `inference.py`: Lightweight inference code that combines dataset loading, model construction, and AP configuration. The script is mainly used for quick testing and as a baseline code.

Detection Visualization.

- `experiments/visualize_detections.py`: The script runs inference on a selected scene and exports predicted and ground truth bounding boxes as PLY files. Predicted boxes are NMS-filtered and coloured by semantic class. The script additionally exports the original mesh.
- `utils/pc_util.py`: Extended with `write_oriented_bbox_cylinder`, `write_ply`, `write_ply_rgb` and `write_ply_color` functions for exporting point clouds and oriented bounding boxes compatible with MeshLab and CloudCompare tools.

Encoder for Classification.

- `datasets/modelnet40.py`: Dataset class for ModelNet40. It loads normal-resampled point clouds, subsamples them to 8192 points and normalizes data. During training, it applies data augmentation (rotation, jitter, scaling), while at test time, it fixes sampling per sample index for reproducibility.
- `models/model_3detr_cls.py`: Class for encoder-only variant of 3DETR for classification task named `Model3DETRCls`. For base encoder, `[CLS]` token is prepended to the input sequence, and its output is passed to a 2-layer MLP classifier. In the masked variant, features are max-pooled. No decoder or object queries are used.
- `engine.py`: The script is extended with `train_one_epoch_cls` and `evaluate_cls` functions implementing the training loop and evaluation of the model through OA and mAcc metrics.
- `main.py`: The script is extended with new `do_train_cls` and `test_model_cls` functions, and also introduces `3detr_cls` as a model option via `--model_name`.

Effect of Number of Queries on Performance and Inference Time.

- `experiments/query_effect_analysis.py`: The script sweeps over different numbers of queries and records AP_{25} and AP_{50} and inference time for each configuration. Results are saved as CSV files.

NMS Effect on Performance.

- `experiments/nms_effect_analysis.py`: The script evaluates AP_{25} and AP_{50} under multiple NMS configurations (no NMS, 2D NMS, 3D class-agnostic NMS and 3D per-class NMS) by reusing same predictions with different `APCalculator` settings. Results are saved as CSV files.

Recall Analysis Across Object Size and Point Count.

- `experiments/binned_recall_analysis.py`: The script computes recall across equal-frequency bins defined by object volume, number of points, or point density, with the bin mode set via `--mode`. Results are saved both as image plots and CSV files.
- `utils/ap_calculator.py`: The script is extended with `compute_recall_by_size_bin`, `compute_recall_by_num_points_bin`, and `compute_recall_by_density_bin` functions, which match predictions to ground truth and compute recall-per-bin at IoU values of 0.25 and 0.5.

Cross-Attention Visualization.

- `experiments/visualize_cross_attention.py`: Runs inference on a selected scene and extracts cross-attention weights from the last three decoder layers. The script colors the point cloud based on attention scores for a set of queries and exports it together with predicted boxes and original mesh as PLY files.